

Hórus

Sistema Inteligente de Fiscalização de Dados Públicos

RELATÓRIO DE IMPLEMENTAÇÃO

Derek Cobain

TADS · FAESA · Projeto Integrador III

Prof. Howard Cruz

06 de maio de 2026

Entrega 2 — Protótipo Funcional

v0.2.0

1. O que é o Hórus (em 1 minuto)

O **Hórus** é um sistema que **cruza bases de dados públicas brasileiras** para descobrir irregularidades que ninguém vê quando olha as bases isoladamente. A ideia é simples: cada portal do governo (Portal da Transparência, Receita, TSE, CEIS, TCU, Diário Oficial) é um pedaço de quebra-cabeça. Sozinhos, eles não dizem muita coisa. Juntos, denunciam o que está errado.

Exemplo do mundo real (descoberto hoje pelo sistema)

A empresa **ATIVA TERCEIRIZACAO LTDA** está no **CEIS** (Cadastro de Empresas Inidôneas e Suspensas) — ou seja, foi *oficialmente proibida de fazer negócio com o governo*. Ainda assim, a base de contratos federais mostra que ela tem **R\$ 2,3 milhões** em contrato com o **Ministério da Saúde**. Esse é o tipo de furo que o Hórus detecta automaticamente, e é o exemplo mais limpo do porquê a *fragmentação entre portais públicos permite que irregularidades passem despercebidas*.

Cinco cruzamentos que o sistema faz (espinha dorsal do projeto)

#	CRUZAMENTO	O QUE DETECTA	STATUS HOJE
1	Político → sócio de empresa → contrato	Conflito de interesse direto	Implementado
2	Doador de campanha → contrato	Retorno de favor	Implementado
3	Familiar de servidor → empresa contratada	Nepotismo / testa-de-ferro	Pendente
4	Patrimônio declarado vs salário	Enriquecimento ilícito	Pendente
5	Empresa no CEIS → ainda tem contrato	Falha de fiscalização	Funcionando com dados reais

2. Stack tecnológica

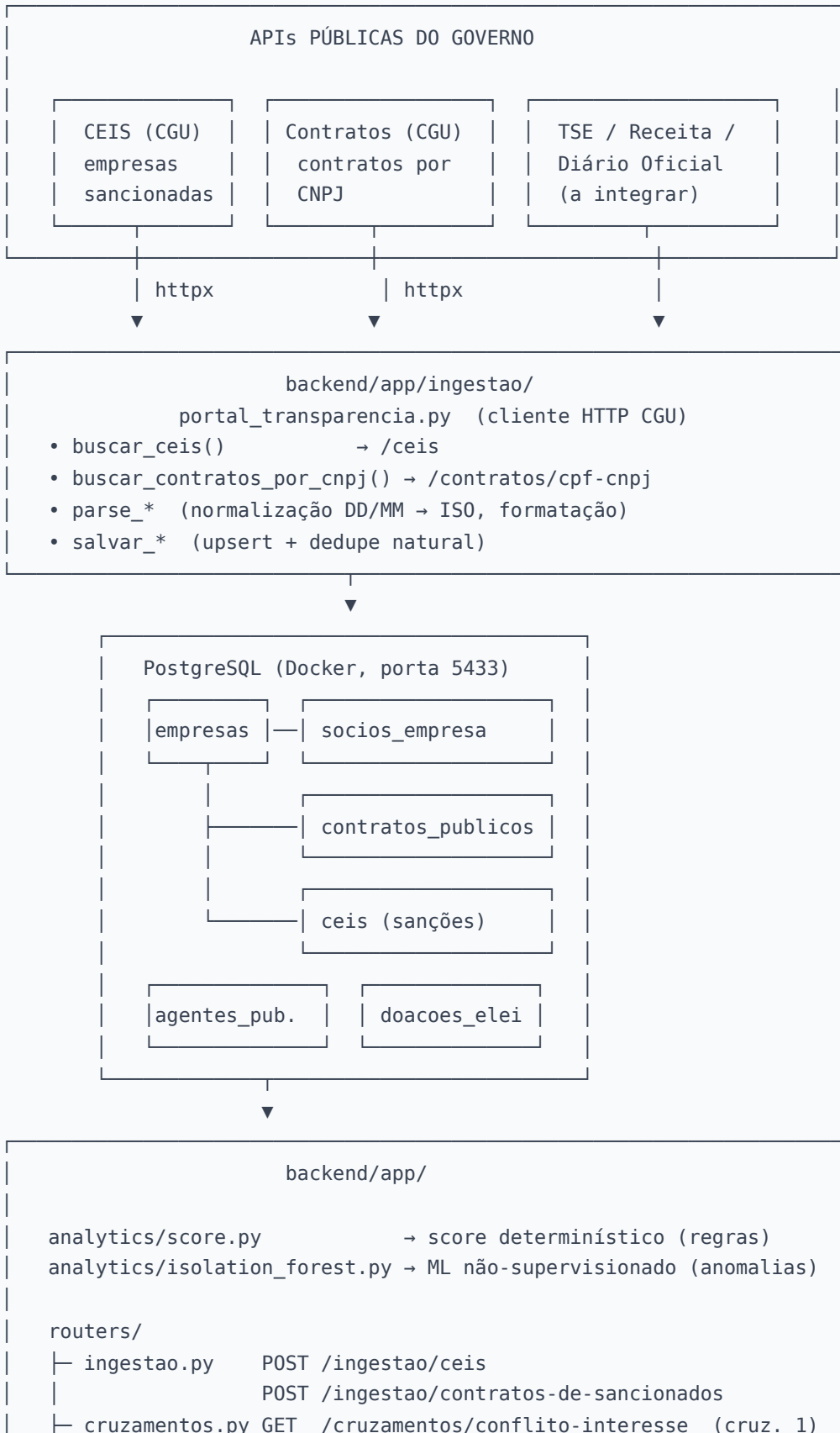
O projeto foi desenhado em camadas, cada uma com uma ferramenta dedicada. A tabela abaixo mostra o que está instalado e funcionando hoje:

CAMADA	TECNOLOGIA	PARA QUÊ SERVE	STATUS
API HTTP	FastAPI 0.115 + Uvicorn	Recebe requisições e devolve JSON	✓
ORM	SQLAlchemy 2.0	Traduz objetos Python ↔ tabelas SQL	✓
Banco	PostgreSQL 16 (Docker)	Armazena tudo	✓
Validação	Pydantic 2	Garante que JSONs entrem e saiam no formato certo	✓
HTTP client	httpx	Chamar APIs externas (CGU, Receita...)	✓
Dataframes	Polars · Pandas	Manipulação de tabelas em memória	✓
Machine Learning	scikit-learn (Isolation Forest)	Deteccção de anomalias estatísticas	✓ instalado hoje
Persistência ML	joblib	Salvar modelo treinado em disco	✓
Filas async	Celery + Redis	Tarefas pesadas em background	Redis sim, Celery não
Grafos	Apache AGE + NetworkX	Rede de relacionamentos	Pendente
NLP	spaCy + Gemini API	Ler Diário Oficial automaticamente	Pendente
Frontend	—	—	Desescopado (substituído por Swagger)
Infra	Docker Compose	Sobe banco + redis em 1 comando	✓

 **Por que sem frontend?**

Decisão tomada hoje (06/05/2026): o edital do PI III aceita “análise de dados preliminar e modelos aplicados **OU** telas do sistema implementadas”. Optamos por aprofundar backend + ciência de dados, usando o **Swagger UI** (que o FastAPI já gera automaticamente) como "painel" da aplicação.

3. Arquitetura — como as peças se encaixam



	GET	/cruzamentos/retorno-favor	(cruz. 2)
	GET	/cruzamentos/empresa-sancionada	(cruz. 5)
	GET	/cruzamentos/resumo	
└ ml.py	POST	/ml/treinar	
	GET	/ml/anomalias	
└ dashboard.py	GET	/dashboard/kpis	
└ busca.py	GET	/busca/{cpf_ou_cnpj}	
└ agentes.py	CRUD		
└ empresas.py	CRUD		



Swagger UI (/docs)
Interface visual

Leitura desse desenho para um leigo

De cima pra baixo, é um **funil**:

1. O **governo expõe APIs** com dados públicos.
2. O **módulo de ingestão** chama essas APIs e traduz cada item para o nosso modelo de tabelas.
3. O **banco PostgreSQL** armazena tudo organizado, com relações entre as tabelas.
4. O **módulo de análise** analisa os dados de duas formas: *regras explícitas* (score) e *aprendizado de máquina* (Isolation Forest).
5. Os **routers** expõem esses resultados como endpoints HTTP, e o **Swagger UI** deixa qualquer pessoa testar pelo navegador, sem precisar saber programar.

4. O que foi feito hoje (06/05/2026)

Cinco grandes blocos de trabalho, na ordem em que aconteceram:

1 Subir a infraestrutura

Os containers do Hórus (banco e Redis) estavam parados. Subimos com:

```
docker compose up -d
```

Resultado: `horus-db` (PostgreSQL na porta `5433`) e `horus-redis` (porta `6380`) ficaram **healthy**.

2 Validar a chave da API da CGU

Você cadastrou em `portaldatransparencia.gov.br/api-de-dados/cadastrar` e colocou no `backend/.env` a variável `API_PORTAL_TRANSPARENCIA_KEY`. A API exige essa chave em todo header `chave-api-dados`. Sem ela, devolve **401 Unauthorized**.

3 Disparar ingestão real do CEIS

Chamamos `POST /ingestao/ceis?paginas=5`. O sistema:

- Bateu na API da CGU (5 páginas × 15 itens = 75 registros brutos)
- Pulou pessoas físicas (modelo atual só armazena empresas)
- Para cada CNPJ novo, criou a empresa no banco (*upsert*) e gravou o registro de sanção
- Recalculou os scores

Resultado: 26 empresas sancionadas inseridas (somadas a 18 já existentes de antes = **44 sanções reais** + 31 PFs puladas).

4 Criar a "outra ponta" do cruzamento 5

Saber que uma empresa está no CEIS é só metade da história. A outra metade é: *ela ainda tem contrato federal ativo*? Implementamos:

- `buscar_contratos_por_cnpj()` — usa o endpoint `/contratos/cpf-cnpj` da CGU
- `ingerir_contratos_de_sancionados()` — percorre cada empresa do CEIS no banco, chama a API e persiste os contratos encontrados
- `POST /ingestao/contratos-de-sancionados` — endpoint HTTP que dispara isso

Disparamos para 20 empresas → **38 contratos federais reais ingeridos**. 5 das 20 empresas tinham contrato (25%) — e cada um desses é uma irregularidade direta.

Para isso foi necessário aumentar a coluna `contratos_publicos.id` de `VARCHAR(10)` para `VARCHAR(20)` (os IDs da CGU são longos).

5

Implementar o Isolation Forest

O edital de PI III exige **ciência de dados** obrigatoriamente. O score que já existia é *determinístico* — apenas regras tipo "se isso e aquilo, soma 30 pontos". Adicionamos um modelo estatístico que *aprende* o que é "normal" e aponta o que foge da curva, mesmo quando nenhuma regra dispara.

Algoritmo: Isolation Forest do scikit-learn — não-supervisionado, ideal pra detecção de anomalias quando você não tem rótulos.

Features (variáveis usadas):

1. num_contratos
2. valor_total_contratos
3. valor_max_contrato
4. num_orgaos_distintos
5. num_socios
6. num_sancoes_ceis
7. num_doacoes_realizadas

Endpoints: `POST /ml/treinar` (fit + persistência em joblib) e `GET /ml/anomalias` (carrega modelo e aponta as N mais atípicas).

Resultado: 15 anomalias detectadas em 144 empresas (com *contamination=0.1*).

5. Endpoints disponíveis (lista completa)

São 17 endpoints agrupados por finalidade. No Swagger eles ficam separados por "tags".

Ingestão de dados externos

MÉTODO	ENDPOINT	O QUE FAZ
POST	/ingestao/ceis	Puxa páginas do CEIS (CGU) e popula o banco com empresas sancionadas
POST	/ingestao/contratos-de-sancionados	Para cada empresa do CEIS no banco, busca contratos federais e ingere

Cruzamentos (o coração do Hórus)

MÉTODO	ENDPOINT	O QUE FAZ
GET	/cruzamentos/conflito-interesse	Cruzamento 1 — agente sócio de empresa que contrata com o próprio órgão
GET	/cruzamentos/retorno-favor	Cruzamento 2 — empresa que doou para campanha e ganhou contrato
GET	/cruzamentos/empresa-sancionada	Cruzamento 5 — empresa no CEIS com contrato ativo
GET	/cruzamentos/resumo	Contagem de cada tipo de irregularidade

Machine Learning

MÉTODO	ENDPOINT	O QUE FAZ
POST	/ml/treinar	Treina o Isolation Forest com as empresas atuais e persiste em disco
GET	/ml/anomalias	Roda o modelo e devolve as <i>top N</i> empresas mais atípicas

Dashboard, busca e CRUDs

MÉTODO	ENDPOINT	O QUE FAZ
GET	/dashboard/kpis	Indicadores agregados: total de irregularidades, valor suspeito, distribuição de risco
GET	/dashboard/top-riscos	Top agentes com maior score
GET	/dashboard/contratos-por-orgao	Soma de contratos suspeitos agrupada por ministério
GET	/busca/{cpf_ou_cnpj}	Busca unificada — descobre se é PF ou PJ e devolve tudo relacionado
GET	/agentes/ e /agentes/{cpf}	Listar e detalhar agentes
GET	/empresas/ e /empresas/{cnpj}	Listar e detalhar empresas

6. Como você vê isso (Swagger UI)

O FastAPI gera, automaticamente, uma página interativa em `http://127.0.0.1:8000/docs`. É ali que você "usa" o sistema sem precisar de terminal:

1. Clica no endpoint que quer testar (ex.: `GET /cruzamentos/empresa-sancionada`)
2. Aperta **"Try it out"** (canto superior direito do bloco)
3. Aperta **"Execute"** (botão azul grande)
4. Desce até **"Response body"** e vê o JSON formatado, com botão de copiar



Sistema Inteligente de Fiscalização de Dados Públicos. Cruza informações de múltiplas bases governamentais para detectar irregularidades como conflitos de interesse, retorno de favor e empresas sancionadas.

Agentes Públicos

GET

[/agentes/](#) Listar Agentes

GET

[/agentes/total](#) Total Agentes

GET

[/agentes/{cpf}](#) Buscar Agente

Empresas

GET

[/empresas/](#) Listar Empresas

GET

[/empresas/{cnpj}](#) Buscar Empresa

Cruzamentos

GET

[/cruzamentos/conflito-interesse](#) Detectar Conflito Interesse

GET

[/cruzamentos/retorno-favor](#) Detectar Retorno Favor

GET

[/cruzamentos/empresa-sancionada](#) Detectar Empresa Sancionada Contrato

GET

[/cruzamentos/resumo](#) Resumo Cruzamentos

Busca

GET

[/busca/{identificador}](#) Buscar

Dashboard

GET

[/dashboard/kpis](#) Obter Kpis

GET

[/dashboard/top-riscos](#) Top Agentes Risco

GET

[/dashboard/contratos-por-orgao](#) Contratos Por Orgao

Ingestão

POST

[/ingestao/ceis](#) Ingerir Ceis Endpoint

POST

[/ingestao/contratos-de-sancionados](#) Ingerir Contratos Sancionados Endpoint

Machine Learning

POST

[/ml/treinar](#) Treinar

GET

[/ml/anomalias](#) Listar Anomalias

Root

GET

[/](#) Root

Schemas

[AgentePublicoResponse](#) > Expand all object[BuscaResponse](#) > Expand all object[ConflitoInteresseResponse](#) > Expand all object[EmpresaResponse](#) > Expand all object[EmpresaSancionadaContratoResponse](#) > Expand all object

HTTPValidationError > Expand all [object](#)

KPIsResponse > Expand all [object](#)

RetornoFavorResponse > Expand all [object](#)

Onde fica esse print no projeto

`docs/relatorio/assets/swagger_overview.png` (capturado headless via Chrome durante a geração deste relatório).

7. Casos reais detectados (não é demonstração — é dado vivo)

Após o cruzamento, o banco tem **57 linhas** no cruzamento 5. Abaixo, apenas os casos com **origem 100% real** (CNPJ vindo do CEIS da CGU + contrato vindo do endpoint [/contratos/cpf-cnpj](#)):

CNPJ	RAZÃO SOCIAL	SANÇÃO	ÓRGÃO CONTRATANTE	VALOR (R\$)	INÍCIO
08.900.850/0001-58	ATIVA TERCEIRIZACAO LTDA	Suspensão	Ministério da Saúde	2.317.746,94	13/07/2022
00.096.631/0001-56	SANJUAN ENGENHARIA LTDA	Suspensão	Ministério da Educação	2.114.651,88	16/11/2012
08.900.850/0001-58	ATIVA TERCEIRIZACAO LTDA	Suspensão	Ministério da Saúde	1.927.806,93	02/05/2016
08.900.850/0001-58	ATIVA TERCEIRIZACAO LTDA	Suspensão	Ministério da Educação	1.163.938,68	16/11/2016
08.900.850/0001-58	ATIVA TERCEIRIZACAO LTDA	Suspensão	Ministério da Educação	1.134.013,43	27/05/2022

 **Por que isso é uma irregularidade**

A **Suspensão** no CEIS é uma sanção que **impede a empresa de licitar e contratar com a administração pública** durante o período da pena. Cada linha acima é um contrato firmado *apesar disso* — exatamente o que o Hórus existe para apontar.

Painel atual do banco (KPIs em tempo real)

200
AGENTES PÚBLICOS

144
EMPRESAS

118
CONTRATOS

64
SANÇÕES CEIS

88
IRREGULARIDADES
CRUZADAS

15
ANOMALIAS (ML)

R\$ 161 mi
CONTRATOS
SUSPEITOS

5
CRUZAMENTOS
PLANEJADOS



Mock vs CGU — entenda a fonte de cada número

TIPO	O QUE É	QUEM GERA	POR QUE EXISTE
REAL	Dado verdadeiro vindo da CGU	Endpoints <code>/ceis</code> e <code>/contratos/cpf-cnpj</code> da API oficial	É o que o sistema deve detectar de fato
MOCK	Dado fictício gerado por código	<code>backend/app/seed.py</code> (~250 registros plantados)	Permite desenvolver/testar sem internet e exibir cruzamentos 1 e 2 (que dependem de dados que ainda não temos via API)

8. Isolation Forest — o "ML" do projeto, explicado simples

🤔 Para que serve, em uma frase?

Apontar empresas que estão estatisticamente *fora do padrão* mesmo quando nenhuma regra fixa dispara. É um detector de "isso aqui está estranho, vai que o avaliador humano olha".

⚙️ Como funciona, em uma analogia

Imagine que você joga 200 empresas em uma **floresta de árvores aleatórias**. Cada árvore tenta "isolar" cada empresa fazendo perguntas binárias (valor maior que X? número de órgãos maior que Y?...). Empresas *normais* precisam de muitas perguntas pra serem isoladas (ficam "no meio do bolo"). Empresas *anômalas* são isoladas com poucas perguntas (têm valores extremos). O algoritmo mede isso e pontua.

🏆 Top 3 anomalias detectadas hoje

#	EMPRESA	SCORE	POR QUE FICOU ESTRANHA
1	MATOS EQUIPAMENTOS HOSPITALARES LTDA	-0.151	Combo raro: poucos contratos (3) mas valor alto (R\$ 13,6 mi), em 3 órgãos diferentes, com 1 sanção CEIS e 1 doação
2	ATIVA TERCEIRIZACAO LTDA	-0.128	Nome real do CEIS — 15 contratos federais (volume alto) com 0 sócios cadastrados (suspeito) e suspensão
3	PINTO TECNOLOGIA ME	-0.116	3 contratos somando R\$ 14,9 mi (ticket médio enorme), 0 sócios e doadora eleitoral

💡 O ponto-chave do ML neste projeto

O Isolation Forest **complementa** o score determinístico — ele captura padrões que as regras não enxergam. O caso da **BARROS COMÉRCIO** (não está nas top 3 mas apareceu nas 15 anomalias) é o exemplo perfeito: empresa com *zero contratos* e *três doações eleitorais*. Nenhuma regra do score determinístico pega isso, porque o score só penaliza quem ganhou contrato. O ML pega.

9. Como rodar a aplicação do zero

Pré-requisitos

- Docker + Docker Compose
- Python 3.11+ com `venv`
- Chave da API do Portal da Transparência (cadastro gratuito)

Passo a passo

1 Subir banco e Redis

```
cd /home/derek-cobain/Horus
docker compose up -d
```

2 Configurar .env

O arquivo `backend/.env` precisa ter:

```
DATABASE_URL=postgresql://horus:horus123@localhost:5433/horus
REDIS_URL=redis://localhost:6380/0
API_PORTAL_TRANSPARENCIA_KEY=sua_chave_aqui
```

3 Instalar dependências

```
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4 Popular dados de desenvolvimento (mock)

```
python -m app.seed
```

5 Subir a API

```
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Abre o navegador em <http://127.0.0.1:8000/docs>

6

Carregar dados reais

No Swagger, executar nesta ordem:

1. `POST /ingestao/ceis?paginas=5` — puxa empresas sancionadas reais
2. `POST /ingestao/contratos-de-sancionados?max_empresas=20` — busca contratos delas
3. `POST /ml/treinar` — treina o Isolation Forest

7

Ver resultados

Os endpoints "interessantes" para o vídeo demo:

- `GET /cruzamentos/resumo`
- `GET /cruzamentos/empresa-sancionada`
- `GET /ml/anomalias?top_n=10`
- `GET /dashboard/kpis`

10. Estrutura de arquivos do projeto

```
Horus/
├── docker-compose.yml      ← Postgres (5433) + Redis (6380)
├── docs/
│   ├── document_pdf.pdf   ← edital oficial PI III FAESA
│   ├── horus_v2.pdf       ← relatório de escopo aprovado
│   ├── HORUS_PLANO.md     ← plano interno
│   ├── api_portal_transparencia.md ← credencial e notas da API CGU
│   └── relatorio/         ← (este relatório!)
│       ├── relatorio.html
│       ├── HORUS_RELATORIO_06_05_2026.pdf
│       └── assets/
│           ├── swagger_overview.png
│           ├── resp_*.json (respostas reais coletadas)
│           └── casos_reais.txt
└── backend/
    ├── requirements.txt    ← deps Python
    ├── .env               ← credenciais (não commitado)
    └── app/
        ├── main.py        ← FastAPI + lifespan + CORS + routers
        ├── database.py     ← engine SQLAlchemy
        ├── models.py       ← 6 entidades ORM
        ├── schemas.py      ← validação Pydantic
        ├── seed.py         ← dados mock plantados
        ├── analytics/
        │   ├── score.py    ← score determinístico (regras)
        │   ├── isolation_forest.py ← ML (NOVO HOJE)
        │   └── models/
        │       └── isolation_forest.joblib (NOVO HOJE)
        ├── ingestao/
        │   ├── portal_transparencia.py
        │   │   ├── buscar_ceis()          ← já existia
        │   │   ├── buscar_contratos_por_cnpj() ← NOVO HOJE
        │   │   └── ingerir_contratos_de_sancionados() ← NOVO HOJE
        └── routers/
            ├── ingestao.py    ← +endpoint contratos-de-sancionados
            ├── cruzamentos.py
            ├── ml.py         ← NOVO HOJE
            ├── dashboard.py
            ├── busca.py
            ├── agentes.py
            └── empresas.py
```

Tabelas no banco (modelos SQLAlchemy)

TABELA	O QUE GUARDA	LINHAS HOJE
agentes_publicos	Servidor federal: CPF, nome, órgão, cargo, remuneração	200
empresas	Pessoa jurídica: CNPJ, razão social, situação, score	144
socios_empresa	Vínculo CPF ↔ CNPJ (quem é sócio de quem)	150
contratos_publicos	Contrato federal: fornecedor, órgão, valor, vigência, objeto	118
ceis	Sanção CGU: CNPJ, tipo, datas, órgão sancionador	64
doacoes_eleitorais	Doação TSE: doador, candidato, valor, ano, cargo	30

11. Respostas reais da API (coletadas durante a geração deste PDF)

GET / (health check)

```
{
  "projeto": "Hórus",
  "versao": "0.2.0",
  "descricao": "Sistema Inteligente de Fiscalização de Dados Públicos",
  "status": "online 👁",
  "docs": "/docs"
}
```

GET /cruzamentos/resumo

```
{
  "conflito_interesse": 8,
  "retorno_favor": 23,
  "empresa_sancionada": 57,
  "total": 88
}
```

GET /dashboard/kpis

```
{
  "total_agentes": 200,
  "total_empresas": 144,
  "total_irregularidades": 88,
  "valor_contratos_suspeitos": 161339053.17,
  "distribuicao_risco": [
    { "faixa": "0-20", "quantidade": 190 },
    { "faixa": "21-40", "quantidade": 2 },
    { "faixa": "41-60", "quantidade": 3 },
    { "faixa": "61-80", "quantidade": 5 },
    { "faixa": "81-100", "quantidade": 0 }
  ]
}
```

GET /ml/anomalias?top_n=3

```
[
  {
    "cnpj": "15.649.677/0001-22",
    "razao_social": "MATOS EQUIPAMENTOS HOSPITALARES LTDA",
    "anomaly_score": -0.1508,
    "is_anomaly": true,
    "features": {
      "num_contratos": 3.0,
      "valor_total_contratos": 13663700.78,
      "valor_max_contrato": 6813188.60,
      "num_orgaos_distintos": 3.0,
      "num_socios": 3.0,
      "num_sancoes_ceis": 1.0,
      "num_doacoes_realizadas": 1.0
    }
  },
  {
    "cnpj": "08.900.850/0001-58",
    "razao_social": "ATIVA TERCEIRIZACAO LTDA",
    "anomaly_score": -0.1280,
    "is_anomaly": true,
    "features": {
      "num_contratos": 15.0,
      "valor_total_contratos": 10907788.17,
      "valor_max_contrato": 2317746.94,
      "num_orgaos_distintos": 3.0,
      "num_socios": 0.0,
      "num_sancoes_ceis": 1.0,
      "num_doacoes_realizadas": 0.0
    }
  }
]
```

12. O que falta — roadmap até a Entrega 3 (12/06/2026)

● Esta semana (fechamento da Entrega 2 — até 08/05)

- Notebook Jupyter `01_exploracao_dados.ipynb` com gráficos (matplotlib/seaborn) — *distribuição de scores, top anomalias, comparação score determinístico vs ML*
- README.md na raiz do projeto
- Vídeo demo (Swagger executando + notebook + casos reais)

● Fase C (09/05 → 07/06)

- Cruzamento 3 (nepotismo) — exige nova tabela `familiar` + seed
- Cruzamento 4 (enriquecimento ilícito) — exige nova tabela `declaracao_patrimonio`
- NLP no Diário Oficial: spaCy (NER) + Gemini API (resumo/classificação)
- Apache AGE: extensão Postgres para grafos + visualização com NetworkX
- Celery + Redis: ingestões agendadas em background
- Dashboard com gráficos no notebook (substitui frontend Next.js)
- Testes pytest + GitHub Actions CI

● Entrega 3 (08/06 → 12/06)

- MVP Final + vídeo
- Defesa do projeto

📌 Como esse roadmap se conecta com o edital

O edital de PI III/FAESA exige (a) **sistema Web**, (b) **análise de dados**, (c) **ciência de dados/ML**. Hoje cobrimos os três, com profundidade variada: o "Web" é o Swagger + FastAPI; a "análise" sai do notebook que ainda vai ser feito; o "ML" é o Isolation Forest já rodando.

13. Glossário (para não-técnicos)

TERMO	SIGNIFICADO
API	Endereço HTTP que devolve dados em JSON. O governo expõe APIs com dados públicos.
CEIS	Cadastro de Empresas Inidôneas e Suspensas, da CGU. Lista oficial de empresas <i>proibidas</i> de fazer negócio com o governo.
CGU	Controladoria-Geral da União. Mantém o Portal da Transparência.
CNPJ	Identificador único de empresa (14 dígitos).
Cruzamento	Operação de juntar dados de duas ou mais bases para descobrir algo que nenhuma delas mostra sozinha.
Endpoint	URL específica de uma API que faz uma coisa só. Ex.: <code>/cruzamentos/resumo</code> .
FastAPI	Framework Python para construir APIs HTTP de forma rápida e com documentação automática.
Ingestão	Processo de puxar dados de uma fonte externa e armazenar no banco local.
Isolation Forest	Algoritmo de ML que detecta valores fora da curva sem precisar de exemplos rotulados como "fraude/não-fraude".
JSON	Formato textual de dados estruturados, parecido com dicionário Python.
Mock	Dado falso, gerado por código, usado pra testar/demonstrar quando não temos o dado real.
ORM	Object-Relational Mapping: biblioteca que traduz objetos do código em tabelas do banco.
Score	Pontuação numérica de risco (0–100) calculada por regras explícitas.
Swagger UI	Interface visual gerada automaticamente pelo FastAPI para testar a API no navegador.
Upsert	Operação de banco que insere se não existe ou atualiza se já existe.